

Applied Generative AI

Session 2 : How Large Language Models Work and Why Prompting Works

Learning Objectives

By the end of this session, you will be able to:

- Explain how Large Language Models process text.
- Understand Tokens, Embeddings, Attention and Transformers.
- Explain how GPT generates responses.
- Differentiate between Training and Inference.
- Understand Emergence and In-Context Learning.
- Write effective prompts using a structured framework.
- Explain why hallucinations occur.
- Understand why Retrieval-Augmented Generation (RAG) exists.

Introduction

When you type a prompt into ChatGPT such as:

Explain Quantum Computing like I'm 10 years old.

the model responds within a few seconds.

It often feels like the AI understands:

- Quantum Computing
- Children
- Language
- Context
- Human reasoning

But does it really understand these concepts?

The answer is No—not in the way humans do.

Large Language Models (LLMs) do not think like people. Instead, they convert language into numbers, identify patterns, calculate probabilities, and generate one token at a time until a complete response is formed.

Understanding this process is the foundation of modern Generative AI.

The Journey of a Prompt

Every prompt follows approximately the same journey inside an LLM.

User Prompt



Tokenizer



Token IDs



Embeddings



Transformer Layers



Attention



Probability Calculation



Next Token Prediction



Repeat



Final Response

Every concept you learn in this session belongs somewhere in this pipeline.

Step 1 – Tokenization

Humans naturally read complete words.

Computers do not.

Suppose you type:

Artificial Intelligence is changing education.

The model first divides this sentence into smaller pieces called tokens.

Example:

Artificial

Intelligence

is

changing

education

.

Depending on the tokenizer, even a single word may be divided into smaller parts.

Example:

transform

ation

These pieces are called **tokens**.

Why Tokens?

Imagine storing every word from every language.

English alone contains hundreds of thousands of words.

Now include:

- Hindi
- Chinese

- Arabic
- Programming languages
- Emojis

The vocabulary becomes enormous.

Instead, LLMs reuse smaller text fragments.

Think of tokens as LEGO blocks.

Just as thousands of structures can be built using LEGO bricks, millions of words can be represented using reusable tokens.

Step 2 – Tokens Become Numbers

Computers cannot calculate using text.

Every token is converted into a numerical identifier.

Example:

Token	ID
Artificial	1245
Intelligence	8271
Education	5632

Now mathematical operations become possible.

However, these numbers do not represent meaning.

They are simply identifiers.

Step 3 – Embeddings

This is where meaning enters the picture.

An **Embedding** is a numerical representation of **meaning**.

Instead of representing a word with a single number, the model represents it using hundreds or even thousands of numerical dimensions.

Words with similar meanings are placed close together.

Example:

King Queen

Prince Princess

while completely different concepts such as:

Pizza

Laptop

Football

occupy different regions of the embedding space.

Mental Model

Think of embeddings as Google Maps coordinates for meaning.

Instead of locating cities, they locate concepts.

Step 4 – Attention

Words rarely have a single meaning.

Consider these two sentences.

I deposited money in the bank.

I sat beside the bank of the river.

Humans instantly know that "bank" has different meanings because of the surrounding words.

Transformers use a mechanism called Attention to do something similar.

Attention answers one important question:

Which words should I focus on while understanding this sentence?

It allows the model to connect related words even when they are far apart.

Without Attention, modern LLMs would not exist.

Step 5 – Transformers

In 2017, Google researchers introduced the Transformer architecture in the paper Attention Is All You Need.

Unlike older models that processed words one after another, Transformers process many relationships simultaneously.

This makes them:

- Faster
- More accurate
- Better at understanding context

Today, nearly every modern LLM is based on the Transformer architecture.

Examples include:

- GPT
- Gemini
- Claude
- Llama
- Mistral

Step 6 – Context Window

Imagine having a conversation with someone who remembers only your last sentence.

Holding a meaningful discussion would be impossible.

LLMs also have memory limits.

This temporary memory is called the **Context Window**.

A larger context window enables a model to:

- Read longer documents
- Understand longer conversations
- Analyse larger codebases
- Process more information at once

This is one reason newer models perform better on complex tasks.

Step 7 – Next Token Prediction

At its core, GPT performs one task repeatedly:

Predict the next token.

Suppose the prompt is:

The capital of France is

The model calculates probabilities.

Token	Probability
Paris	92%
London	2%
Berlin	1%

Rome	1%
------	----

The most probable token is selected.

The sentence becomes:

The capital of France is Paris.

The same process repeats until the response is complete.

Everything—from essays and code to conversations—is generated using this simple process.

Why Does GPT Feel Intelligent?

This is one of the most fascinating questions in AI.

During training, GPT processed:

- Books
- Research papers
- Source code
- Websites
- Articles
- Documentation

Over trillions of tokens, it learned patterns in:

- Grammar
- Programming
- Mathematics
- Writing
- Logic

No one explicitly programmed these skills.

They emerged from learning patterns in data.

This phenomenon is called **Emergence**.

Pattern Learning, Not Memorisation

Many people believe GPT stores answers.

It does not.

Instead, it stores patterns.

Think of a student learning multiplication.

Initially, they memorise many examples.

Eventually, they understand the rule itself.

Similarly, LLMs compress vast amounts of information into patterns that can be applied to new situations.

Why Prompting Works

If GPT only predicts the next token, why do prompts matter?

Because the prompt determines the direction of generation.

Consider these prompts:

Prompt 1

Explain Machine Learning.

Prompt 2

You are a university professor. Explain Machine Learning to a first-year engineering student using simple language, real-world examples, and diagrams.

The second prompt provides more context, resulting in a more useful response.

Good prompts guide the model towards better reasoning paths.

In-Context Learning

One of the most remarkable abilities of LLMs is adapting from examples provided within the prompt.

Example:

Dog → Animal

Rose → Plant

Car → Vehicle

Laptop →

The model predicts:

Device

No retraining occurs.

The model simply recognises the pattern in the prompt.

This ability is called **In-Context Learning**.

Types of Prompting

Zero-Shot Prompting

No examples are provided.

Translate:

Hello

One-Shot Prompting

One example is provided.

Good Morning → Bonjour

Good Night →

Few-Shot Prompting

Several examples are provided.

Dog → Animal

Rose → Plant

Car → Vehicle

Laptop →

Few-shot prompting often improves accuracy because it demonstrates the expected pattern.

The RTCC Framework for Better Prompts

Professional prompts usually contain four elements:

Component	Purpose
Role	Who the AI should act as
Task	What the AI should do
Context	Background information
Constraints	Rules or limitations

Example:

Role:

Senior Python Developer

Task:

Review this FastAPI project.

Context:

It is a REST API for an e-commerce platform.

Constraints:

Suggest only performance improvements.

This structure produces more consistent and useful responses.

Training vs Inference

Training	Inference
Model learns from data	Model generates responses
Takes months	Takes seconds
Uses thousands of GPUs	Happens when users interact with the model
Updates model parameters	Does not change the model

Hallucinations

- LLMs generate the most probable continuation of text.
- They do not verify facts automatically.
- If asked about an event that never happened, the model may confidently produce a fictional answer.

This is called a **hallucination**.

Developers reduce hallucinations by providing reliable context, using external knowledge, or implementing Retrieval-Augmented Generation (RAG).

Why RAG Exists

Suppose a company asks:

What is our leave policy?

The model may not know because the document was never included during training.

Instead of relying only on the model's memory, we retrieve the relevant document first.

Question

↓

Retriever

↓

Relevant Documents

↓

LLM

↓

Grounded Response

This approach is called Retrieval-Augmented Generation (RAG) and will be covered in detail in a later session.

Key Takeaways

After completing this session, you should remember:

1. LLMs process tokens, not words.
2. Tokens are converted into numerical embeddings.
3. Embeddings represent meaning rather than identity.
4. Attention enables models to focus on relevant context.
5. Transformers power almost every modern LLM.
6. GPT generates responses through next-token prediction.
7. Intelligence emerges from learning patterns, not memorising answers.
8. Prompting works because it guides the model's reasoning path.
9. In-Context Learning allows models to adapt without retraining.
10. Hallucinations occur because LLMs predict likely text rather than guaranteed truth.
11. RAG improves responses by grounding them in external knowledge.

What's Next?

In Session 3, we will move from concepts to implementation by learning how to interact with LLMs using APIs, Python, and structured outputs. You will build your first AI-powered application and understand how developers integrate LLMs into real-world software systems.

OpenEd Labs